Creating CI/CD pipelines in Google Cloud Platform (GCP) involves using Google Cloud's suite of tools such as Cloud Source Repositories, Cloud Build, and Cloud Deploy. Here's a step-by-step guide:

Step 1: Set Up Your Environment

1. **Create a GCP Account**: If you don't have a GCP account, [create one here](https://cloud.google.com/).

2. **Create a Project**:
   - Go to the GCP Console.
   - Click the project dropdown and select "New Project."
   - Name your project and click "Create."

3. **Enable Billing**: Ensure billing is enabled for your project.

4. **Enable Required APIs**: Enable the following APIs:
   - Cloud Build API
   - Cloud Deploy API
   - Artifact Registry API (if using for storing build artifacts)

Step 2: Create a Cloud Source Repository

1. **Go to Cloud Source Repositories Console**:
   - Click "Create Repository."
   - Name your repository and click "Create."

2. **Clone the Repository**:
   ```bash
   gcloud source repos clone YOUR_REPO_NAME --project=YOUR_PROJECT_ID
   cd YOUR_REPO_NAME
   ```

```

### Step 3: Create a Cloud Build Configuration

1. **Create a `cloudbuild.yaml` file** in the root of your repository. This file defines your build steps.

   ```yaml
   steps:
   - name: 'gcr.io/cloud-builders/npm'
     args: ['install']
   - name: 'gcr.io/cloud-builders/npm'
     args: ['run', 'build']
   artifacts:
     objects:
       location: 'gs://YOUR_BUCKET_NAME/build-artifacts/'
       paths: ['**']
   ```

2. **Push Your Code**:

   ```bash
   git add .
   git commit -m "Initial commit"
   git push origin master
   ```

Step 4: Set Up Artifact Registry (Optional)

1. **Create a Docker Repository** (if you're building Docker images):

   ```bash
   gcloud artifacts repositories create YOUR_REPO_NAME --repository-format=docker --location=YOUR_REGION
```
```

```

2. **Update `cloudbuild.yaml`** to include Docker build steps:

   ```yaml
   steps:
   - name: 'gcr.io/cloud-builders/docker'
     args: ['build', '-t', 'YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME', '.']
   - name: 'gcr.io/cloud-builders/docker'
     args: ['push', 'YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME']
   ```

Step 5: Create a Cloud Build Trigger

1. **Go to Cloud Build Triggers Console**:
   - Click "Create Trigger."
   - Name your trigger and select your repository.
   - Set the trigger type to "Push to a branch" and specify the branch (e.g., `master`).
   - Specify the build configuration file (`cloudbuild.yaml`).

Step 6: Deploy Using Cloud Deploy

1. **Create a Delivery Pipeline**:
   - Go to Cloud Deploy Console.
   - Click "Create Pipeline."
   - Define the pipeline stages and targets.

2. **Create a Delivery Configuration File**:
   - Create a `clouddeploy.yaml` file in your repository.
   ```yaml

```
apiVersion: deploy.cloud.google.com/v1beta1
kind: DeliveryPipeline
metadata:
  name: example-pipeline
stages:
  - name: dev
    targetId: dev-target
    deploy:
      containers:
        - image: YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME
```

3. **Update Cloud Build Configuration to Include Cloud Deploy**:

```yaml
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME']
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  entrypoint: 'gcloud'
  args: ['deploy', 'releases', 'create', 'RELEASE_NAME', '--delivery-pipeline=example-pipeline', '--region=YOUR_REGION', '--images=YOUR_IMAGE_NAME=YOUR_REGION-docker.pkg.dev/YOUR_PROJECT_ID/YOUR_REPO_NAME/YOUR_IMAGE_NAME']
```

Step 7: Trigger the Pipeline

1. **Push Code**:

- Push code to your repository. This should trigger the Cloud Build trigger automatically.

2. **Monitor the Pipeline**:

- Go to the Cloud Build and Cloud Deploy consoles to monitor the execution of your build and deploy stages.

By following these steps, you can set up a CI/CD pipeline in GCP, integrating Cloud Source Repositories, Cloud Build, and Cloud Deploy. You can customize and extend this pipeline based on your specific requirements and integrate additional services like Cloud Functions, App Engine, and more.